Three-Plane Architecture (operationalization target)

> **Status:** target architecture for Shaw operationalization. Reflects the director's guidance: split standard functions (MCP tool servers) from orchestration (agentic harness), and centralize data-plane complexity on the enterprise standard. The current MVP collapses all three planes into one FastAPI service + local Postgres for prototyping velocity; this document describes the target so the migration path is explicit.
>
> Companion to `system_boundary.md` and `architecture.md`. The boundary contract in those documents is unchanged.

---

1. Why three planes

The v3.0 system boundary diagram draws the world as **inside Ignition** vs. **outside Ignition**. That boundary is correct and load-bearing — control authority stops at the gateway. But the *outside* side has internal structure that matters for IT operationalization:

1. **Agentic harness** — orchestration only. Owns request lifecycle, anchor parsing, phase routing, RCA chain control, response validation. Stateless. Provider-agnostic about every downstream dependency.
2. **MCP tool servers** — deterministic functions, wrapped behind the Model Context Protocol so they are discoverable, callable, and reusable by *any* model client (this advisor today, other Shaw AI projects tomorrow).
3. **Data plane** — relational tables, vector search, ML feature store, immutable audit. The platform IT has standardized on (Databricks / enterprise RDBMS / etc.).

Splitting these three concerns has three concrete benefits:

- **Each plane can be operationalized independently.** The agentic harness can run on one approved host; the tool servers on another; the data plane on the platform IT has already invested in.
- **The MCP tool layer becomes a Shaw asset, not a Coater 1 asset.** Any future Shaw AI project that needs `percentile_of` against the historical distribution gets it for free.
- **The data plane is no longer special.** It is just "the enterprise data platform with vector capability" — whatever Shaw says that is.

---

2. The three planes

2.1 Ignition (OT, read-only) — unchanged

Same as `system_boundary.md` §2. Native Jython, native gateway APIs, no external dependencies. Curated context construction and identity stay here.

2.2 Agentic harness (IT, orchestration only)

The harness owns the **three-phase request lifecycle** but delegates *every* deterministic computation to MCP tools and *every* persistence concern to the data plane.

```
handle_chat(curated_context, query, user)
  ├── Phase 1 (pre-LLM)
  │     ├── parse_anchor(query)              ← in-process (cheap, no I
  │     ├── if control verb        → refuse + audit
  │     ├── if ambiguous           → clarify
  │     ├── retrieval              → MCP tool: hybrid_retrieve
  │     ├── change_ledger          → MCP tool: change_ledger
  │     ├── anomaly_check          → MCP tool: anomaly_check
  │     ├── rules_eval             → MCP tool: rules_eval
  │     ├── line_memory_lookup     → MCP tool: memory_search
  │     └── if insufficient        → templated refusal + audit
  │
  ├── Phase 2 (LLM)
  │     ├── two-step RCA chain (orchestrated here)
  │     │     ├── step 1: hypothesise (LLM, no tools)
  │     │     └── step 2: adjudicate (LLM + MCP tools, bounded budget)
  │     └── response_parser
  │           ├── enforce citations
  │           ├── strip uncited claims
  │           └── apply confidence label
  │
  └── Phase 3 (persist)
        ├── data_plane.write_message(snapshot)
        ├── data_plane.append_audit(hash_chain)
        └── data_plane.intake_feedback_hooks()
```

What lives in the harness, and *only* the harness:

- Anchor parsing and control-verb refusal logic.
- Phase routing (when to short-circuit, when to call the model, when to run the RCA chain instead of single-shot RAG).
- Tool-call orchestration and budget enforcement.
- Response parsing, citation enforcement, confidence labeling.
- Hash-chain construction (the cryptographic discipline; the *storage* is the data plane).

What does **not** live in the harness:

- SQL. No direct DB calls. Everything goes through `data_plane`.
- Deterministic math. No `percentile_of`, no drift detection, no distribution comparison written here. All MCP tools.
- Model invocation specifics. One adapter, one config switch.

2.3 MCP tool servers (IT, reusable)

Every "standard function/method" — pure, read-only, deterministic — is exposed as an MCP tool. Today's tools, mapped:

Tool	What it computes
`percentile_of`	Empirical CDF lookup for a tag value, scoped by recipe
`compare_to_distribution`	Percentile + nearest-runs + outcome summary

`nearest_historical_runs`	k nearest production runs by fingerprint
`detect_drift`	Page-Hinkley CUSUM drift check on a tag's recent series
`defect_events_in_window`	Bounded read of defect events with grouping
`change_ledger`	What changed since baseline (recipe, crew, equipment)
`anomaly_check`	Multivariate Mahalanobis check on the curated tag block
`rules_eval`	Evaluate declarative YAML rules against a tag snapshot
`hybrid_retrieve`	Vector + BM25 + RRF + MMR over the document corpus
`memory_search`	Read approved line-memory entries by similarity

Properties every tool inherits by being an MCP server:

- **Discoverable** — clients enumerate the tool list and JSON schemas.
- **Callable from any model client** — not coupled to this harness.
- **Independently versionable** — bumping a tool's schema is a server release, not a harness release.
- **Independently observable** — each tool server emits its own metrics.
- **Independently governable** — Shaw can audit, allowlist, or sandbox each tool server separately.

Hosting model is IT's call: one MCP server per tool, one server per *family* of tools, or one server hosting all of them. The harness only needs the server URL(s) and credentials.

2.4 Data plane (IT, enterprise standard)

The data plane is whatever Shaw has standardized on (likely Databricks). The harness sees it through one adapter (`service/db/data_plane.py`) that exposes:

- Relational reads/writes against the schema in `data_model.md`.
- Vector similarity search (used by the `hybrid_retrieve` tool).
- ML feature snapshots (used by anomaly model refit jobs).
- Append-only audit with platform-enforced immutability.
- Scheduled jobs (outcome closure, anomaly refit, WO sync, ingestion).

What the data plane must support, regardless of platform:

1. Relational tables matching `data_model.md`.
2. k-NN vector similarity over chunk and memory embeddings.
3. Trigram/BM25-style keyword search.
4. Append-only audit table with platform-enforced UPDATE/DELETE block.
5. Partitioning + retention on high-volume tables.
6. Scheduled job runtime.

If the chosen platform splits these across two systems (e.g. RDBMS for OLTP + Vector Search service for retrieval), the adapter handles the fan-out. The harness does not know.

---

3. Migration path from MVP to three-plane

The MVP collapses all three planes into one FastAPI service + local PostgreSQL. The migration is incremental and does not require a rewrite.

Step	Action	Status
1	Stand up the data-plane adapter `service/db/data_plane.py` so harness no longer talks to a specific DB driver	scaffolded — see §4
2	Wrap deterministic tools as an MCP server (`service/mcp/`)	scaffolded — see §5
3	Switch harness to call the local MCP server in-process	next sprint
4	Point data-plane adapter at the Shaw-approved store	post-IT-meeting
5	Promote MCP server to its own host	post-IT-meeting
6	Decommission local Postgres	once parity verified

Each step is independently testable. Steps 1–2 are reversible (the harness can fall back to direct calls). Steps 3–6 are the actual operationalization.

---

4. The data-plane adapter

Single seam. See `service/db/data_plane.py`.

The adapter is a typed protocol that the harness depends on. Today it has one implementation (`PostgresDataPlane`). Future implementations (`DatabricksDataPlane`, etc.) plug in behind the same protocol with no changes upstream.

The point is **not** to abstract away SQL prematurely. The point is to make the boundary explicit so the IT meeting can answer "where does this plane live?" without us having to point at a hundred call sites.

---

5. The MCP tool server scaffold

See `service/mcp/`. The scaffold:

- Reuses the existing tool implementations in `service/services/tools.py` — no logic duplication.
- Exposes them via the MCP protocol (transport-agnostic — stdio for local development, HTTP/SSE for hosted deployment).
- Is independently runnable: `python -m mcp.server`.
- Has a deliberately thin handler layer; all behavior lives in the underlying functions.

This is the *evidence* you bring to the IT meeting that the split is real, not a slide. "The tool server is already running locally; we need a host for it that other Shaw AI projects can also reach."

---

6. What stays the same across all stacks

- The boundary contract (Ignition ↔ outside) in `system_boundary.md` §4.
- The three-phase request lifecycle.

- Refusal-first behavior on insufficient evidence.
- Mandatory citations with downgrade-on-no-citation.
- Confidence labels.
- Audit immutability.
- Provider-agnostic LLM adapter.

These are the architectural contracts. Everything else is implementation.